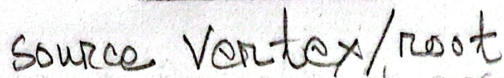


DSA-1
Graph-II

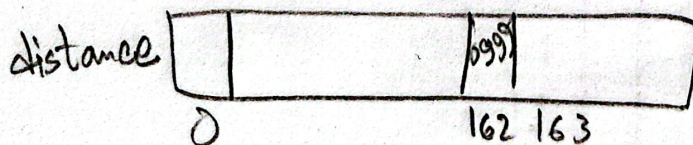
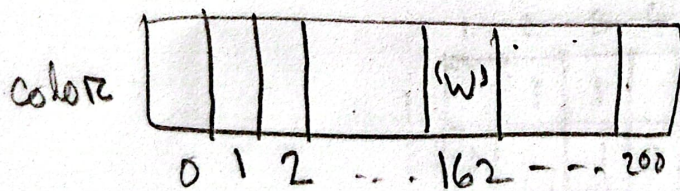
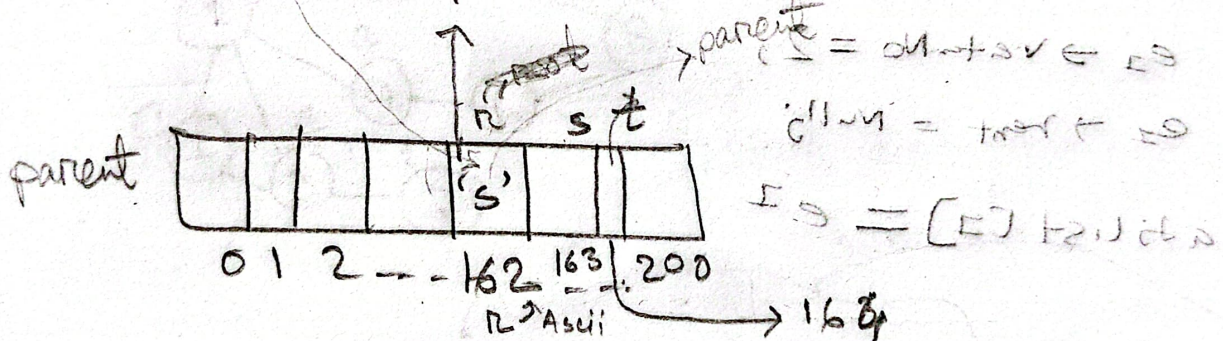


parent
distance
color

white color $\rightarrow$ এছাড়া touch করা হয় নি, $\neq$
gray color $\rightarrow$ যে Node এ $\neq$ ভ্রমণ traverse করা
হলো। $\therefore [white] \& touch \& touch$

Black $\rightarrow$ Node is connected
8 $\downarrow$ all nodes gray color.

parent[pr] = 's'



→ source
 (S)

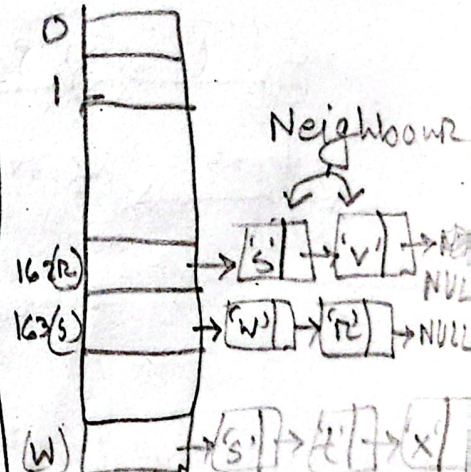
Q (S)

d | 0 |
 R S t

→ निम्नलिखित दृष्टिकोण से विचार करें zero

C | W | 8 | W | W | W | W

slide - 25

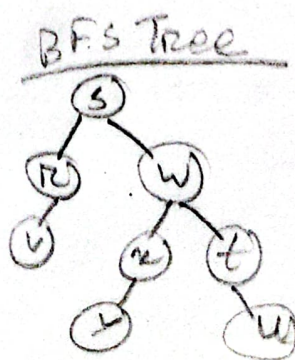
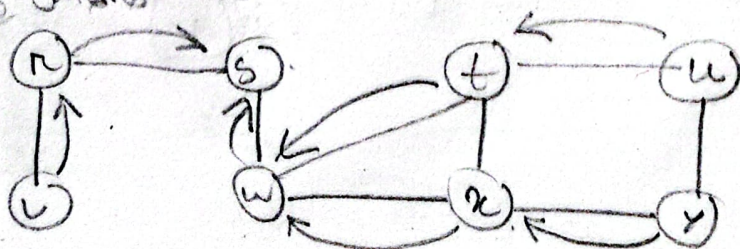


→ source
 * S दृष्टिकोण से shortest path?

parent P P
 y → R → W → S

Array से Box दृष्टिकोण में

* Tree दृष्टिकोण में



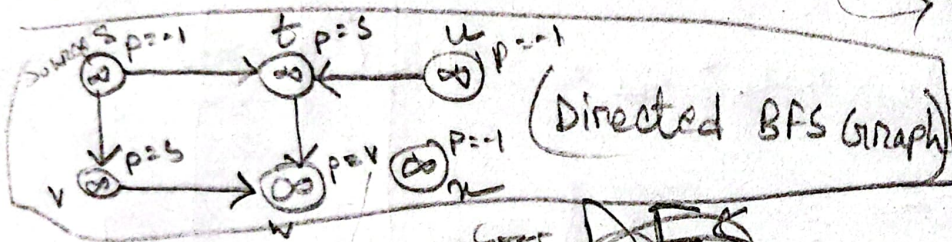
BFS() [pseudocode] → exam এ আসবে
 নাহ, তাহলে
 pseudo code
 আসবে নাহ
 এটা ফাংশন

```

  {
    → }
  
```

$Q \rightarrow \emptyset$
 → empty Queue

while ($Q \neq \emptyset$)
 → Queue যতক্ষণ empty
 হচ্ছে না।



DFS

→ এটা রিসিউস
 ফাংশন।
 এর p

DFS

Initialization
 time = 1

for each vertex v in the graph
 DFS_Visit(v)

DFS_Visit: start

color[v] = gray // v.color = gray
time++

set discovery time [v] // set v.d

given graph state

(initial state) for each adj vertex u of v

if (u.color is white)

DFS_Visit(u)

time++

set finish time [v] // set v.f

color[v] = black // v.color = black

graph or graph state of graph will

Stack use DFS

initialization; time = 0

stack ← S; S.color = gray;

S.d = 4 + time;

while stack is not empty

u ← Pop

for each adj vertex v of u

if v is white

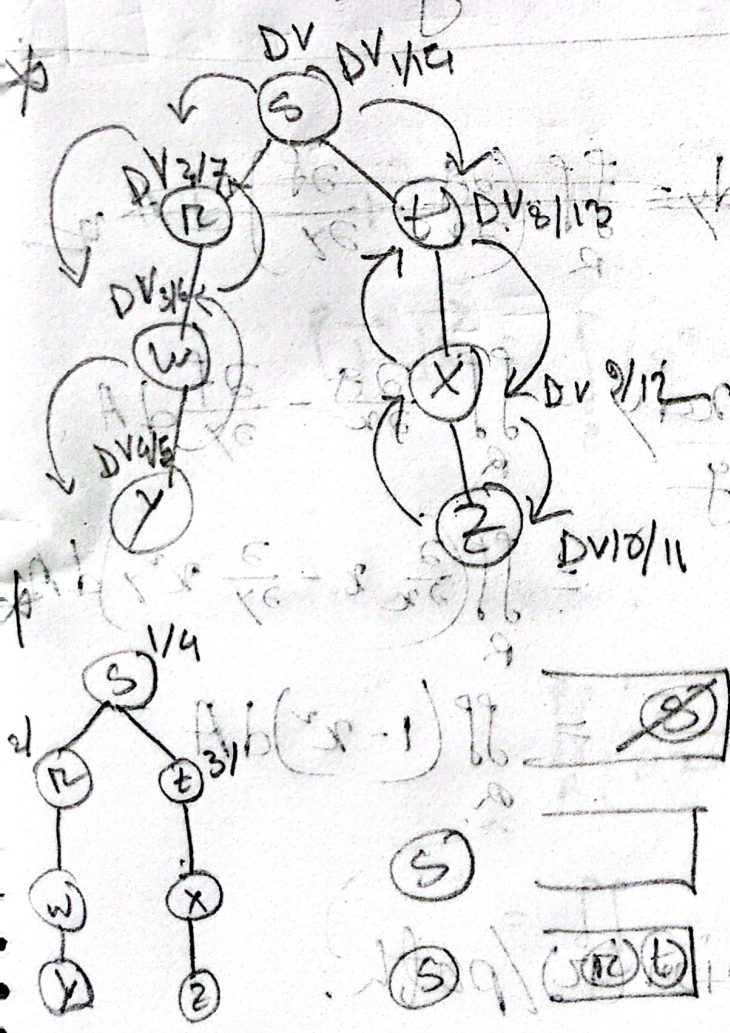
v.color = gray;

time++

V.d = time

push(v);

u.color = black; time++



(S)

(S)

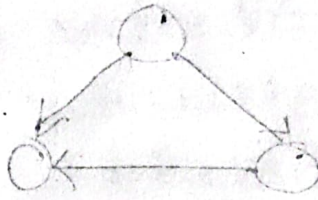
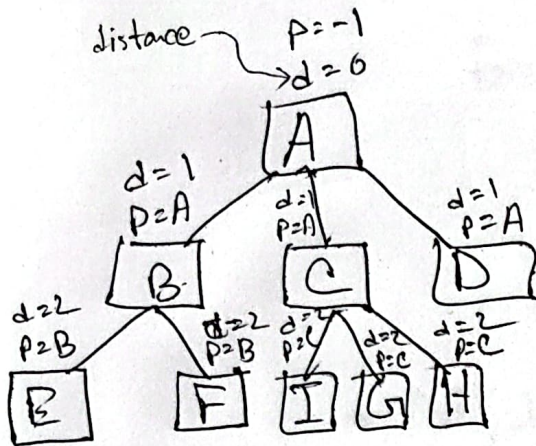
(S)

(R) (T)

DSA-1

Level order traversal

CT-5 syllabus
Graph-I
Graph-II
(only BFS, DFS)



Level order traversal

Queue: A

A B C D Console: A

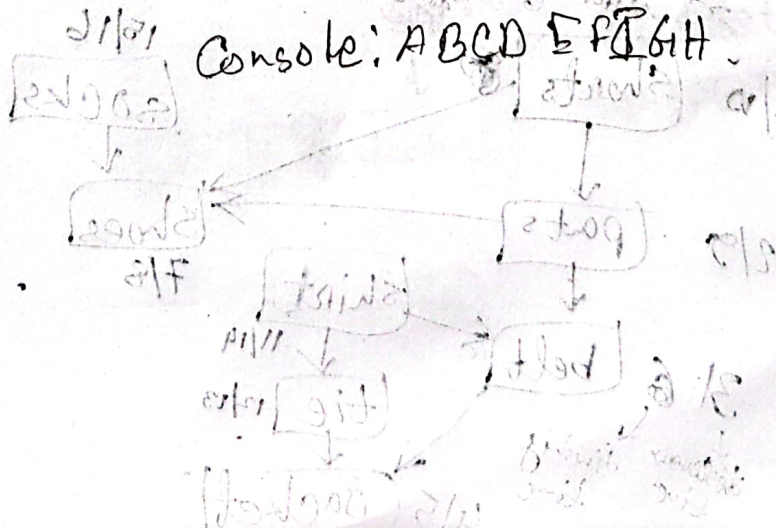
A B C D E F Console: AB

A B C D E F G H Console: ABC

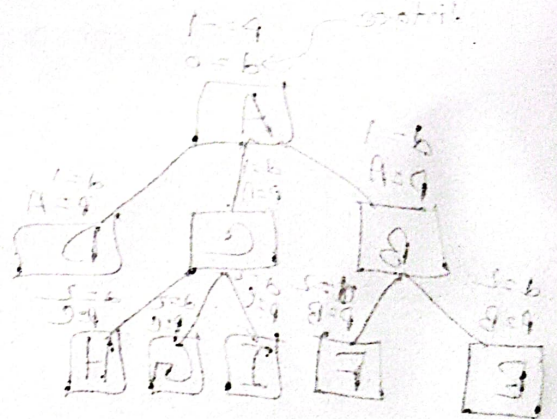
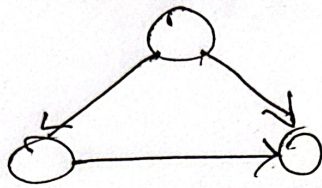
A B C D E F G H Console: ABCD

Console: ABCD E F G H

if (queue.isEmpty()) return;

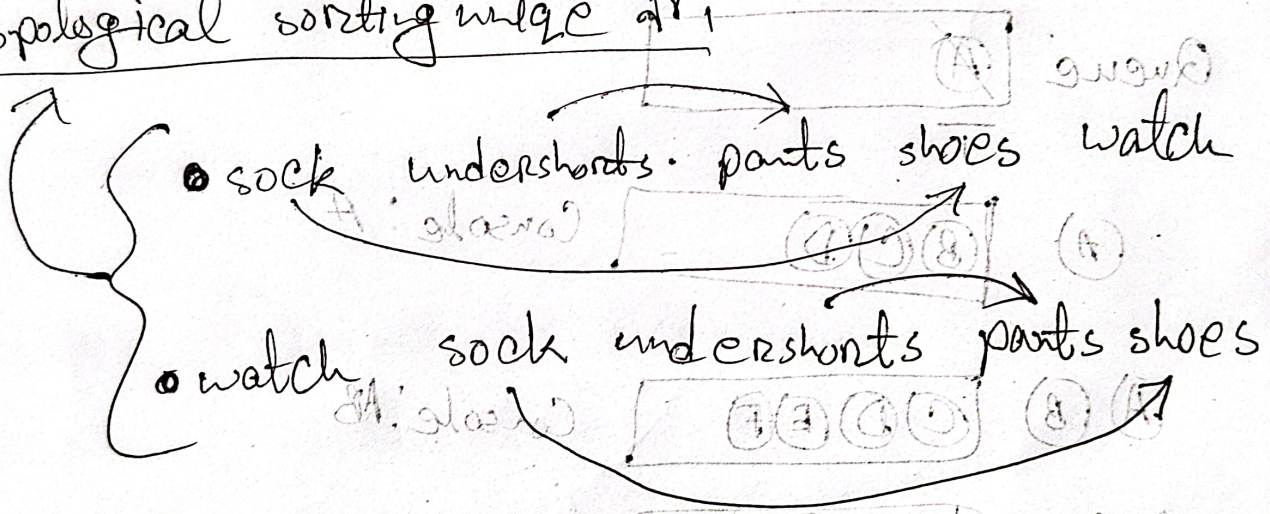


Directed Acyclic Graphs

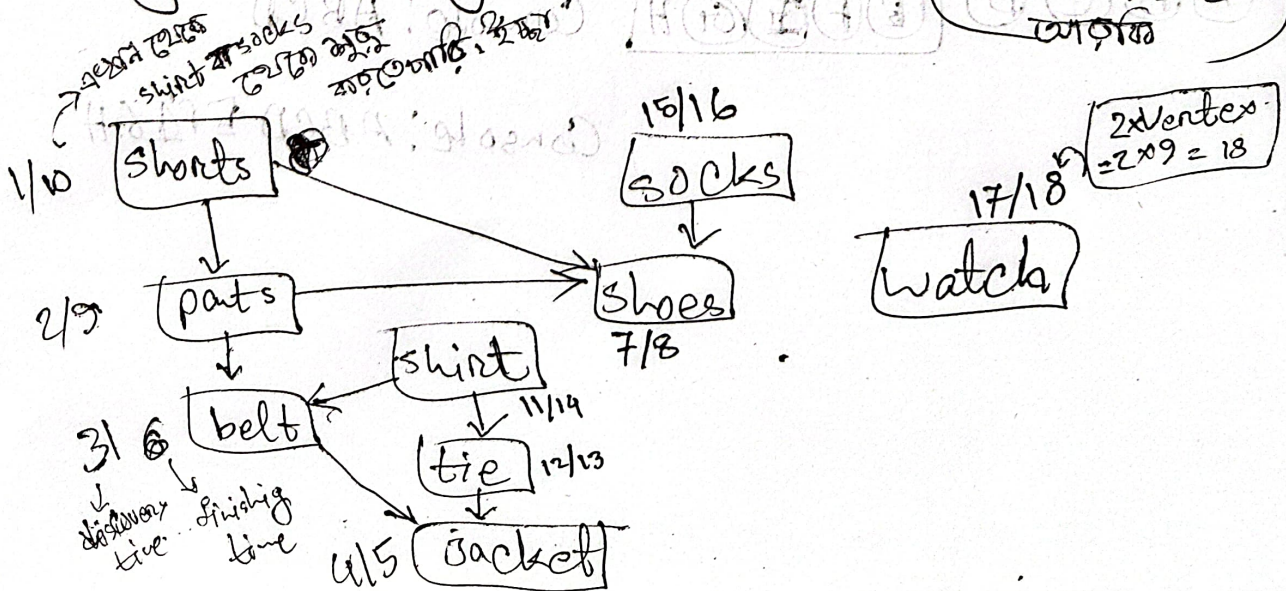


Topological Sort

Topological sorting using array



* Topological sorting stack based (DFS is stack based)



~~DFS~~

• DFS()

Initialization $\rightarrow$ time = 0, colors = 'w';

for each vertex $v \in G$:

if v is white:

DFS_Visit(v);

• DFS_Visit(v)

v .color = 'g';

discover
time
~~time~~

time = time + 1;

$v.d = \text{time}$;

for each vertex u adj to v :

if u is white:

DFS_Visit(u);

time = time + 1;

finishing
time

$v.f = \text{time}$;

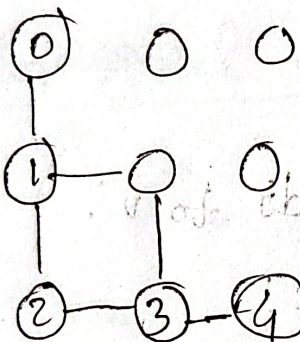
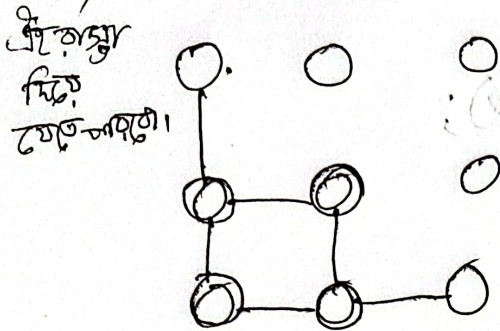
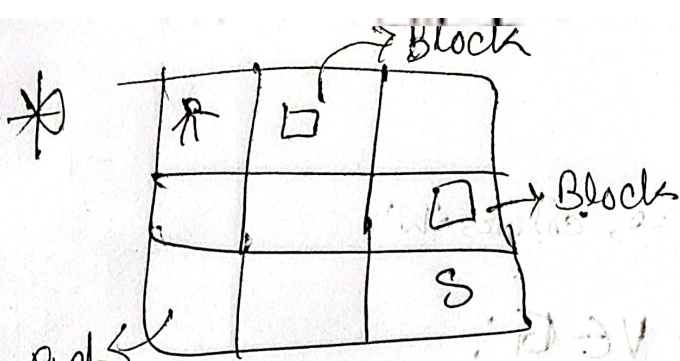
v .color = Black;

Finishing: 5 6 8 9 10 13 14 16 18

shorts $\rightarrow$ pants $\rightarrow$ shoes $\rightarrow$ belt $\rightarrow$ Jacket

watch $\rightarrow$ socks $\rightarrow$ shirt $\rightarrow$ tie

Time Complexity: $O(V + E)$; same as DFS



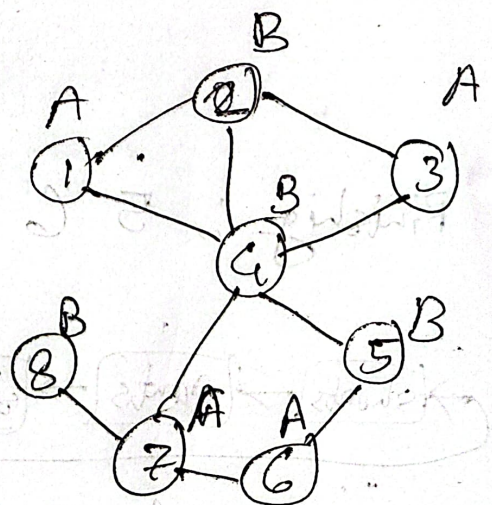
Disjoint Set

Not for final
for Ds A-11

$$\text{Graph} = (V, E)$$

$$V = \{1, 2, 3, \dots, 8\}$$

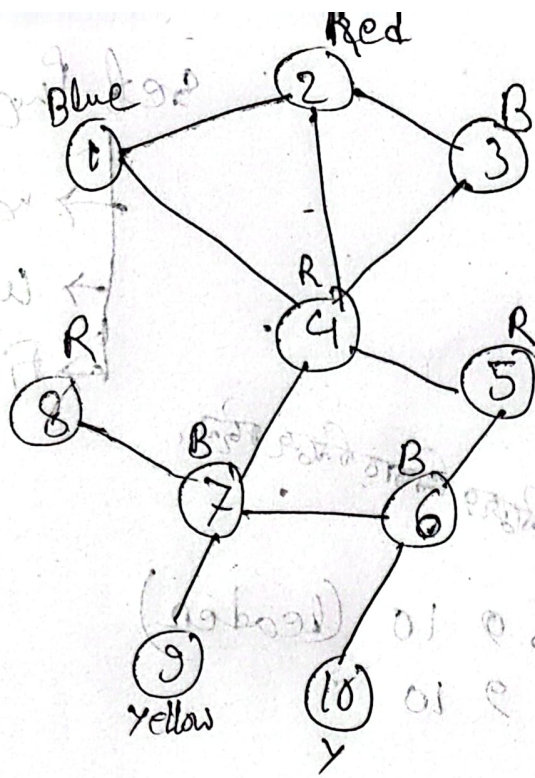
$$E = \{(1, 2), (2, 3), (5, 6), \dots\}$$



$$A = \{1, 3, 6, 7\}$$

$$B = \{2, 4, 5, 8\}$$

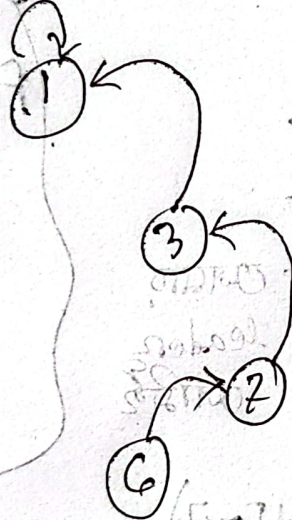
Sub set



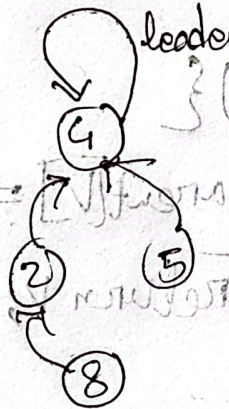
color: 'b' 'r' 'b' 'r' 'r' 'b' 'b' 'r' 'y' 'y'

color: 'b' 'r' 'b' 'r' 'r' 'b' 'b' 'r' 'y' 'y'
1 2 3 4 5 6 7 8 9 10

leader 2



leader 4



leader 9



* parent: 1 2 3 4 5 6 7 8 9 10

make set

leader	1	2	3	4	5	6	7	8	9	10
Array	1	2	3	4	5	6	7	8	9	10

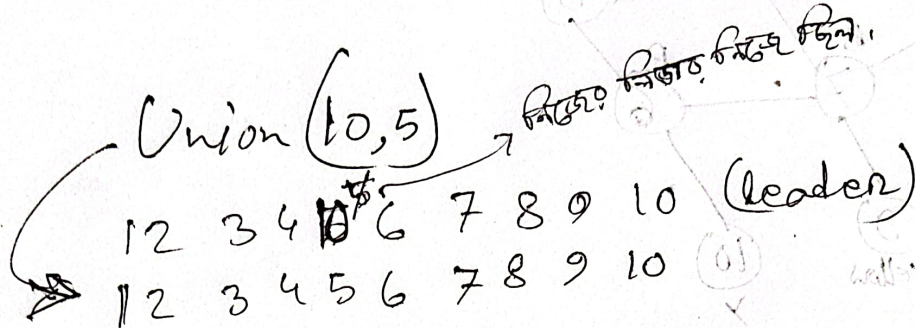
→ array में प्रत्येक निम्नलिखित
निम्नलिखित leader.

Union

Union(a, b)

set function

- make set()
- union()
- findset()



Find Set

FindSet(v) {

if (parent[v] == v) {

return v;

→ 5tho daktar or leader to

}

else {

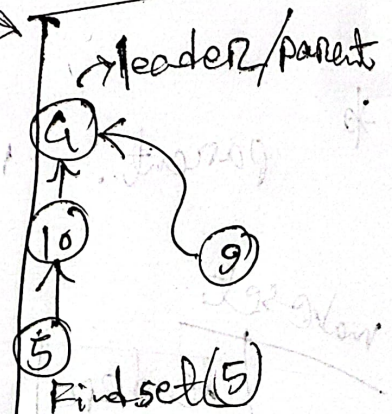
x = FindSet(parent[v])

return x;

}

FindSet(v)

→ 5tho daktar or leader to



	2	3	4	5	6	7	8	9	10
leader	2	1	4	10	6	7	8	9	4

Time Complexity : $O(n)$ (worstcase)